MINI PROJECT REPORT

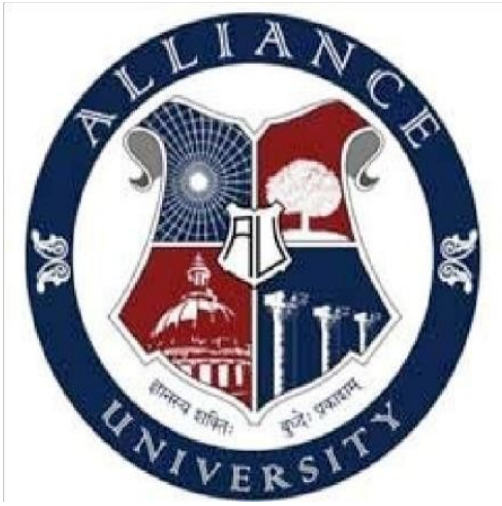
INSTRUCTION CYCLE SIMULATION ON A CUSTOM CPU

Submitted by:
YOUR FULL NAME
YOUR ROLL NUMBER

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING

Under the guidance of
COA FACULTY NAME
Assistant Professor, ASAC



Alliance School Of Advance Computing

Alliance University, Bengaluru

October-2026



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on the Instruction Cycle and Addressing Modes (RegexCPU) is submitted by YOUR FULL NAME [Uni. Roll No. YOUR ROLL NUMBER] in partial fulfilment for the award of the degree of Bachelor Of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026. This report embodies the results of original work and studies conducted by students and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

COA FACULTY NAME
Assistant Professor



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled INSTRUCTION CYCLE SIMULATION ON A CUSTOM CPU, submitted by me in partial fulfilment of the requirements for the award of the degree of **Bachelor Of Technology in Computer Science and Engineering (CSE)** at **Alliance University**, is a record of my original work carried out under the supervision of **COA FACULTY NAME, Assistant Professor, ASAC**.

I further confirm that all views, findings, and conclusions expressed herein are the result of sincere effort and academic inquiry, and have been reviewed and deliberated in consultation with the assigned mentor.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

YOUR FULL NAME	YOUR ROLL NUMBER	
Name of the Student	University Registration Number	Signature

Acknowledgement

I would like to express my heartfelt gratitude to everyone who contributed to the successful completion of this project titled INSTRUCTION CYCLE SIMULATION ON A CUSTOM CPU.

First and foremost, I extend my sincere thanks to my mentor and guide, COA FACULTY NAME, for the support, insightful feedback, and encouragement throughout the development of this work. Their expertise and commitment to fostering learning provided the foundation for this project.

I am deeply grateful to my faculty members and peers whose guidance and technical inputs helped refine the project. Their collaborative spirit and constructive suggestions were instrumental in shaping the final outcome.

This project allowed me to build a CPU model in which the instruction cycle is visible and measurable, and to run a real program on it. It strengthened my understanding of registers, addressing modes and CPI.

Lastly, I would like to thank everyone who supported me directly or indirectly during this journey.

Thank you.

YOUR FULL NAME

TABLE OF CONTENTS

Acknowledgement.....**4**

Abstract.....**6**

Chapter I- Introduction.....**7**

Problem Statement.....7

Theory.....8

Chapter II - System Configuration.....**9**

Algorithm.....10

Source Code.....10

Chapter III - Results.....**14**

Chapter IV – Conclusion.....**19**

Abstract

The instruction cycle (fetch, decode, execute) is the heartbeat of every stored-program computer. This report presents a small Von-Neumann CPU simulator that models the instruction cycle at micro-operation level (PC, MAR, MDR, IR, flags), a 20-instruction instruction set with immediate, direct, register and indexed addressing, a two-pass assembler, and a code generator that compiles a minimised DFA into a table-driven pattern-matching program. The simulator runs this program on test strings and reports instructions executed, clock cycles and cycles per instruction (CPI).

Experiments show that the generated matcher executes exactly 11 instructions (58 clock cycles) per input symbol, so total cost follows cycles = 41 + 58n with CPI close to 5.27. The CPU's accept/reject decisions agreed with Python's `re` module on 3,000 randomly generated regex/string pairs. The CPU is the Computer Organization and Architecture part of RegexCPU, a combined project that also covers Formal Languages and Automata Theory (regex to automata) and Design and Analysis of Algorithms (Tarjan's SCC algorithm).

Chapter I- Introduction

A stored-program computer keeps instructions and data in the same memory and executes a program by repeating one fixed routine, the **instruction cycle**: fetch the next instruction from memory, decode it, execute it and move on. Every feature of a processor, from registers and buses to addressing modes and pipelines, exists to make this routine correct and fast. Lecture diagrams of the cycle are easy to read but hard to internalise, because the student never sees a real program change real registers one clock tick at a time.

This work builds a small CPU in software so that the cycle can be observed and measured. The CPU is not a toy exercise with a hand-written five-line program: it runs a program that is **generated automatically** from a regular expression. A regular expression is compiled to a minimal deterministic finite automaton (the FLAT part of the project), analysed with Tarjan's strongly connected components algorithm (the DAA part), and the resulting transition table is loaded into CPU memory together with a machine-code matcher that walks the table using **indexed addressing**. The simulator then reports how many instructions and clock cycles the match took.

Problem Statement

Textbook descriptions of the instruction cycle, registers and addressing modes stay abstract unless a student can run a real workload and inspect every register transfer. The problem addressed in this report is to design and implement a processor model in which the instruction cycle is explicit, and to use it to execute a non-trivial program while measuring its cost. The objectives are:

- define an instruction set architecture (ISA) with a fixed 32-bit format and several addressing modes;
- implement a two-pass assembler that turns assembly text into machine words;
- implement a CPU whose `step()` performs fetch, decode, execute and memory access with visible micro-operations;
- generate machine code for a DFA pattern matcher automatically and run it on test strings;
- measure instruction count, clock cycles and CPI, and verify the result against independent matchers.

Theory

Stored-program (Von Neumann) model

Instructions and data share one address space. The processor contains a **program counter (PC)** holding the address of the next instruction, a **memory address register (MAR)** and **memory data register (MDR)** that connect it to memory, and an **instruction register (IR)** holding the instruction being executed. The model used here follows this organisation exactly: one 32,768-word memory, one set of eight general-purpose registers and the four special registers.

The instruction cycle

Table 1: Phases of the instruction cycle implemented by CPU.step()

Step	Micro-operation	Purpose
T0	MAR ← PC	put the address of the next instruction on the address bus
T1	MDR ← M[MAR]; PC ← PC + 1	read the instruction word and advance the PC
T2	IR ← MDR	latch the instruction
T3	decode IR	split into opcode, rd, rs and immediate / address
T4	execute	ALU operation, comparison, branch or I/O
T5	memory access	only for LD, LDX and ST: operand read or write

Instruction format and addressing modes

Every instruction is one 32-bit word: `opcode (8 bits) | rd (4) | rs (4) | imm/address (16, signed)`. The ISA supports four addressing modes. In **immediate** mode the operand is inside the instruction (`LDI R0, 5`). In **direct** mode the instruction holds the memory address (`LD R1, 100`). In **register** mode the operand is in a register (`MOV R2, R0`). In **indexed** mode the effective address is a register plus a constant (`LDX R0, R2, table` computes `EA = R2 + table`), which is the natural way to index a two-dimensional array stored in a flat memory.

Performance equation

Execution time = Instruction count × CPI × Clock period. The simulator reports the first two terms directly. Because all clock periods are equal in a simulation, comparing programs by instructions and cycles is sufficient.

Chapter II - System Configuration

The simulator was developed and tested in the environment below. It needs nothing beyond a standard Python installation to run the CPU; Flask is used only for the web demonstration.

Table 2: Development and execution environment

Item	Configuration
Operating system	Linux 6.18.44-fc-v153
Processor	Intel(R) Xeon(R) Processor @ 2.80GHz
CPU cores	1
Memory (RAM)	3.9 GB
Language / runtime	Python 3.12.3
Libraries	Flask (web app), matplotlib + numpy (graphs); core engines use only the Python standard library
Testing	unittest (28 tests), Python `re` as reference oracle

Simulated CPU configuration

Table 3: Simulated processor

Component	Specification
Architecture	Von Neumann, single memory for code and data
Memory	32,768 words × 32 bits (16-bit addresses)
General registers	R0–R7, 32-bit signed
Special registers	PC, MAR, MDR, IR; flags Z (zero) and N (negative)
Instruction word	32 bits: opcode 8 | rd 4 | rs 4 | imm/addr 16 (signed)
Addressing modes	immediate, direct, register, indexed
Input device	`IN rd` reads the next input symbol class (−1 at end of input)
Cycle model	fetch 3 + decode 1 + execute 1 = 5; +1 memory operand; +2 multiply

Table 4: Instruction set of the simulated CPU

Mnemonic	Opcode	Operands	Addressing	Cycles
NOP	0	–	–	5
LDI	1	rd, imm	immediate	5
LD	2	rd, imm	direct	6
LDX	3	rd, rs, imm	indexed	6
ST	4	rs, imm	direct	6
MOV	5	rd, rs	register	5
ADD	6	rd, rs	register	5
ADDI	7	rd, imm	immediate	5
MUL	8	rd, rs	register	7
MULI	9	rd, imm	immediate	7
CMP	10	rd, rs	register	5
CMPI	11	rd, imm	immediate	5
JMP	12	imm	–	5

JEQ	13	imm	–	5
JNE	14	imm	–	5
JLT	15	imm	–	5
IN	16	rd	–	5
ACCEPT	17	–	–	5
REJECT	18	–	–	5
HLT	19	–	–	5

Algorithm

Instruction-cycle algorithm (CPU.step)

1. MAR ← PC; MDR ← M[MAR]; PC ← PC + 1; IR ← MDR (fetch).
2. Split IR into opcode, rd, rs, imm; stop with an error if the opcode is undefined (decode).
3. Perform the operation: update a register, set Z/N, change PC for a taken branch, read or write memory, or halt (execute).
4. Add the instruction's cost to the cycle counter and increment the instruction counter.
5. Repeat until ACCEPT, REJECT or HLT sets the halt flag, or until the cycle limit is exceeded.

Generated DFA-matcher program

For a DFA with K symbols, state s and symbol class c, the transition table is stored row-major at address `table` so that the next state is `M[table + s·K + c]`. Registers: R0 = state, R1 = symbol, R2 = table index, R3 = accept flag.

6. R0 ← start state.
7. R1 ← next input symbol (`IN`). If R1 = −1 go to step 7. If R1 < −1 (symbol outside the alphabet) REJECT.
8. R2 ← R0 · K (`MULI`).
9. R2 ← R2 + R1 (`ADD`).
10. R0 ← M[R2 + table] (`LDX`, indexed addressing).
11. If R0 = −1 (dead transition) REJECT, otherwise jump to step 2.
12. R3 ← accept[R0] (`LDX`). ACCEPT if R3 = 1, else REJECT.

Source Code

Instruction encoding and decoding

```
def encode(op: str, rd: int = 0, rs: int = 0, imm: int = 0) -> int:
    if not -32768 <= imm <= 32767:
        raise CPUError(f"immediate/address {imm} does not fit in 16 bits")
    return (OPCODES[op] << 24) | (rd << 20) | (rs << 16) | (imm & 0xFFFF)

def decode(word: int):
    op = (word >> 24) & 0xFF
```

```
    rd, rs = (word >> 20) & 0xF, (word >> 16) & 0xF
    imm = word & 0xFFFF
    if imm & 0x8000:
        imm -= 0x10000
    return op, rd, rs, imm
```

The instruction cycle: CPU.step()

```
def step(self, trace: bool = False):
    if self.halted:
        raise CPUError("CPU is halted")
    micro = []
    pc0 = self.pc
    # FETCH
    self.mar = self.pc
    self.mdr = self._read(self.mar)
    self.pc += 1
    self.ir = self.mdr
    if trace:
        micro += [f"T0  MAR <- PC            (MAR = {pc0})",
                  f"T1  MDR <- M[MAR]; PC++  (MDR = {self.mdr:#010x}, PC =
{self.pc})",
                  f"T2  IR  <- MDR"]
    # DECODE
    op, rd, rs, imm = decode(self.ir)
    if op not in MNEMONIC:
        raise CPUError(f"illegal opcode {op} at PC={pc0}")
    name = MNEMONIC[op]
    if trace:
        micro.append(f"T3  DECODE  opcode={name} rd=R{rd} rs=R{rs} imm={imm}"
                     + (f"  [{ADDRESSING[name]} addressing]" if name in ADDRESSING
else ""))
    # EXECUTE (+ memory access / write-back)
    R = self.regs
    if name == "NOP":
        detail = "no operation"
    elif name == "LDI":
        R[rd] = imm
        detail = f"R{rd} <- {imm}"
    elif name == "LD":
        self.mar = imm
        self.mdr = self._read(self.mar)
        R[rd] = to_signed32(self.mdr)
        detail = f"MAR <- {imm}; MDR <- M[{imm}]; R{rd} <- {R[rd]}"
    elif name == "LDX":
        ea = R[rs] + imm
        self.mar = ea
        self.mdr = self._read(ea)
        R[rd] = to_signed32(self.mdr)
        detail = f"EA = R{rs}({R[rs] if rs != rd else '..'}) + {imm} = {ea}; R{rd}
<- M[{ea}] = {R[rd]}"
    elif name == "ST":
        ea = imm
        if not 0 <= ea < self.mem_words:
            raise CPUError(f"memory write out of range: {ea}")
        self.mem[ea] = R[rs]
        detail = f"M[{ea}] <- R{rs} ({R[rs]})"
    elif name == "MOV":
        R[rd] = R[rs]
        detail = f"R{rd} <- R{rs} ({R[rs]})"
    elif name in ("ADD", "ADDI"):
        b = R[rs] if name == "ADD" else imm
        R[rd] = to_signed32(R[rd] + b)
        detail = f"R{rd} <- R{rd} + {b} = {R[rd]}"
    elif name in ("MUL", "MULI"):
        b = R[rs] if name == "MUL" else imm
        R[rd] = to_signed32(R[rd] * b)
        detail = f"R{rd} <- R{rd} * {b} = {R[rd]}"
    elif name in ("CMP", "CMPI"):
        b = R[rs] if name == "CMP" else imm
        diff = R[rd] - b
```

```
        self.z, self.n = diff == 0, diff < 0
        detail = f"R{rd}({R[rd]}) - {b} -> Z={int(self.z)} N={int(self.n)}"
    elif name == "JMP":
        self.pc = imm
        detail = f"PC <- {imm}"
    elif name in ("JEQ", "JNE", "JLT"):
        taken = {"JEQ": self.z, "JNE": not self.z, "JLT": self.n}[name]
        if taken:
            self.pc = imm
        detail = f"{'taken' if taken else 'not taken'}: PC = {self.pc}"
    elif name == "IN":
        if self.inp_pos < len(self.inp):
            R[rd] = self.inp[self.inp_pos]
            self.inp_pos += 1
        else:
            R[rd] = -1
        detail = f"R{rd} <- input symbol {R[rd]}"
    elif name in ("ACCEPT", "REJECT", "HLT"):
        self.halted = True
        self.status = name if name != "HLT" else "HALT"
        detail = f"halt, status = {self.status}"
    cost = CYCLES[name]
    self.cycles += cost
    self.instructions += 1
    self.mix[name] = self.mix.get(name, 0) + 1
    if not trace:
        return None
    micro.append(f"T4  EXECUTE {detail}")
    if name in ("LD", "LDX", "ST"):
        micro.append("T5  MEMORY ACCESS completed")
    return {
        "pc": pc0, "word": self.ir, "asm": disassemble(self.ir), "micro": micro,
        "regs": list(self.regs), "z": int(self.z), "n": int(self.n),
        "cost": cost, "cycles": self.cycles,
    }
```

Machine-code generator for a DFA

```
def dfa_to_assembly(dfa, title="") -> str:
        from .dfa import to_table
    table, accept = to_table(dfa)
    k = len(dfa.alphabet)
    lines = [
        f"; ---- DFA matcher {('for ' + title) if title else ''}".rstrip(),
        f"; alphabet = {dfa.alphabet}   states = {dfa.n}   K = {k}",
        f"        LDI   R0, {dfa.start}        ; R0 = start state",
        "loop:   IN    R1              ; R1 = next symbol class (-1 end, -2
unknown)",
        "        CMPI  R1, -1",
        "        JEQ   done            ; end of input",
        "        JLT   reject          ; symbol not in alphabet",
        "        MOV   R2, R0",
        f"        MULI  R2, {k}              ; R2 = state * K",
        "        ADD   R2, R1          ; R2 = state * K + symbol",
        "        LDX   R0, R2, table   ; indexed addressing: R0 = M[R2 + table]",
        "        CMPI  R0, -1",
        "        JEQ   reject          ; dead transition",
        "        JMP   loop",
        "done:   LDX   R3, R0, accept  ; R3 = accept[state]",
        "        CMPI  R3, 1",
        "        JEQ   yes",
        "reject: REJECT",
        "yes:    ACCEPT",
    ]
    if table:
        for i in range(0, len(table), 16):
            lines.append(("table:  " if i == 0 else "        ") + ".word " + ",
".join(map(str, table[i:i + 16])))
    else:
        lines.append("table:  .word 0")
    lines.append("accept: .word " + ", ".join(map(str, accept)))
```

```
    return "\n".join(lines) + "\n"
```

Chapter III - Results

All measurements below were produced by the project itself (`main.py`, `benchmarks/benchmark.py` and the unit tests). The reference pattern is `(a|b)*abb`, whose minimal DFA has four states.

Generated program and memory map

The pattern compiles to 17 instruction words followed by 12 data words (an 8-entry transition table and a 4-entry accept table).

Table 5: Memory image of the matcher

Address	Word (hex)	Meaning
0	01000000	LDI R0, 0
1	10100000	IN R1
2	0b10ffff	CMPI R1, -1
3	0d00000c	JEQ 12
4	0f00000f	JLT 15
5	05200000	MOV R2, R0
6	09200002	MULI R2, 2
7	06210000	ADD R2, R1
8	03020011	LDX R0, R2, 17
9	0b00ffff	CMPI R0, -1
10	0d00000f	JEQ 15
11	0c000001	JMP 1
12	03300019	LDX R3, R0, 25
13	0b300001	CMPI R3, 1
14	0d000010	JEQ 16
15	12000000	REJECT
16	11000000	ACCEPT
17	00000001	.word 1
18	00000000	.word 0
19	00000001	.word 1
20	00000002	.word 2
21	00000001	.word 1
22	00000003	.word 3
23	00000001	.word 1
24	00000000	.word 0
25	00000000	.word 0
26	00000000	.word 0
27	00000000	.word 0
28	00000001	.word 1

Instruction-cycle trace

The first instructions of a run, as printed by `python main.py "(a|b)*abb" -t ab --trace`:

```
[000] LDI   R0, 0           cost=5  cycles=5
      T0  MAR <- PC           (MAR = 0)
```

```
      T1  MDR <- M[MAR]; PC++  (MDR = 0x01000000, PC = 1)
      T2  IR  <- MDR
      T3  DECODE  opcode=LDI rd=R0 rs=R0 imm=0  [immediate addressing]
      T4  EXECUTE R0 <- 0
      R = [0, 0, 0, 0, 0, 0, 0, 0]  Z=0 N=0
[001] IN    R1               cost=5  cycles=10
      T0  MAR <- PC            (MAR = 1)
      T1  MDR <- M[MAR]; PC++  (MDR = 0x10100000, PC = 2)
      T2  IR  <- MDR
      T3  DECODE  opcode=IN rd=R1 rs=R0 imm=0
      T4  EXECUTE R1 <- input symbol 0
      R = [0, 0, 0, 0, 0, 0, 0, 0]  Z=0 N=0
[002] CMPI  R1, -1           cost=5  cycles=15
      T0  MAR <- PC            (MAR = 2)
      T1  MDR <- M[MAR]; PC++  (MDR = 0x0b10ffff, PC = 3)
      T2  IR  <- MDR
      T3  DECODE  opcode=CMPI rd=R1 rs=R0 imm=-1  [immediate addressing]
      T4  EXECUTE R1(0) - -1 -> Z=0 N=0
      R = [0, 0, 0, 0, 0, 0, 0, 0]  Z=0 N=0
[003] JEQ   12               cost=5  cycles=20
      T0  MAR <- PC            (MAR = 3)
      T1  MDR <- M[MAR]; PC++  (MDR = 0x0d00000c, PC = 4)
      T2  IR  <- MDR
      T3  DECODE  opcode=JEQ rd=R0 rs=R0 imm=12
      T4  EXECUTE not taken: PC = 4
      R = [0, 0, 0, 0, 0, 0, 0, 0]  Z=0 N=0
[004] JLT   15               cost=5  cycles=25
```

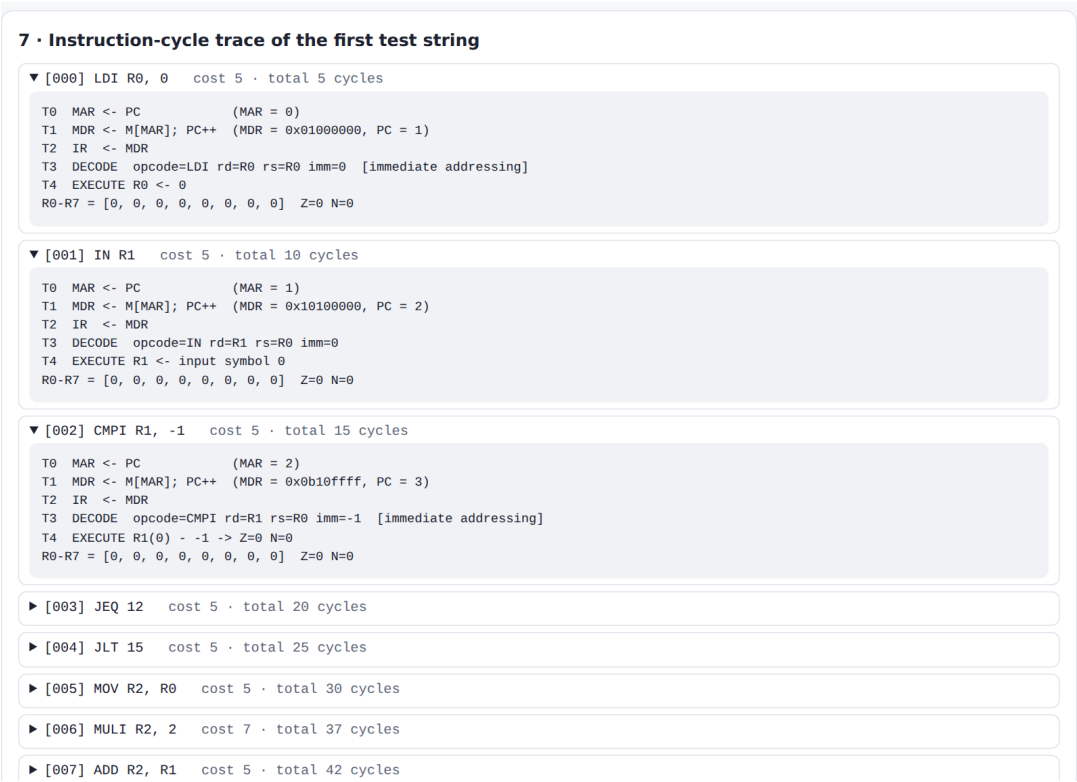


Figure 1: Instruction-cycle trace in the web application (first instructions expanded)

Test strings

Table 6: Results for pattern `(a|b)*abb`

Input	NFA	DFA	Min DFA	CPU	Instr.	Cycles	CPI	Agree
`abb`	accept	accept	accept	accept	41	215	5.244	✔️
`aabb`	accept	accept	accept	accept	52	273	5.25	✔️
`babb`	accept	accept	accept	accept	52	273	5.25	✔️
`ababb`	accept	accept	accept	accept	63	331	5.254	✔️
`ab`	reject	reject	reject	reject	30	157	5.233	✔️

abba	reject	reject	reject	reject	52	273	5.25	✔
ε (empty)	reject	reject	reject	reject	8	41	5.125	✔
abbc	reject	reject	reject	reject	39	204	5.231	✔

For every string the NFA, DFA, minimal DFA and the CPU gave the same verdict. An input symbol outside the alphabet (c) is mapped to class −2 and rejected by the `JLT` instruction.

Cost model: 11 instructions, 58 cycles per symbol

Table 7: Cost of one iteration of the matcher loop

Instruction	Times per symbol	Cycles each	Cycles
IN	1	5	5
CMPI	2	5	10
JEQ	2	5	10
JLT	1	5	5
MOV	1	5	5
MULI	1	7	7
ADD	1	5	5
LDX	1	6	6
JMP	1	5	5
Total	**11**		**58**

Table 8: Measured cycles versus the model cycles = 41 + 58n

Input length n	Instructions	Cycles measured	41 + 58n	CPI
0	8	41	41	5.125
10	118	621	621	5.263
50	558	2941	2941	5.271
100	1108	5841	5841	5.272
150	1658	8741	8741	5.272
200	2208	11641	11641	5.272

The measured values equal the model exactly, which confirms that the cost is linear in the input length and that the start-up and finishing code (LDI, final IN, test, table lookup and ACCEPT) costs 41 cycles. The average CPI approaches 58/11 = 5.27 for long inputs; it is above the minimum of 5 because the multiply (7 cycles) and the table load (6 cycles) are slower.

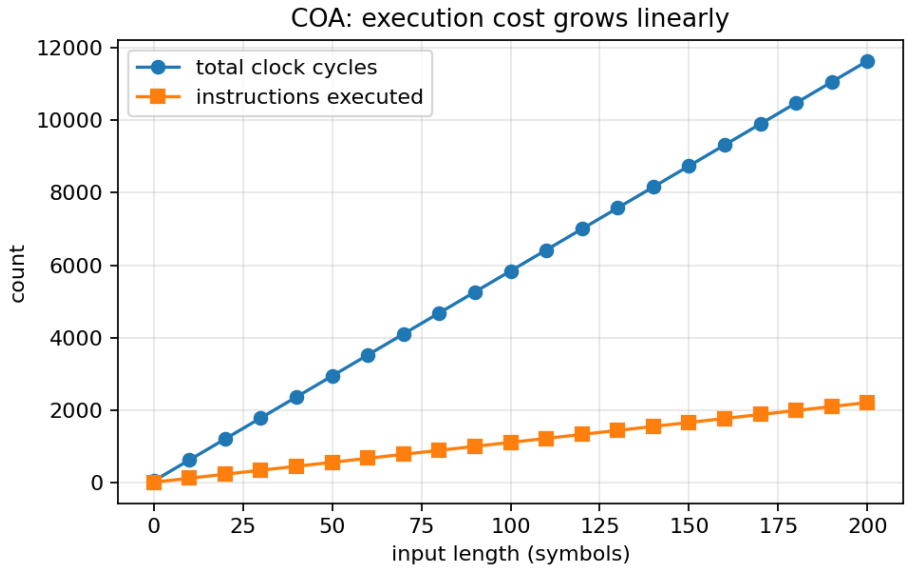


Figure 2: Instructions and clock cycles versus input length

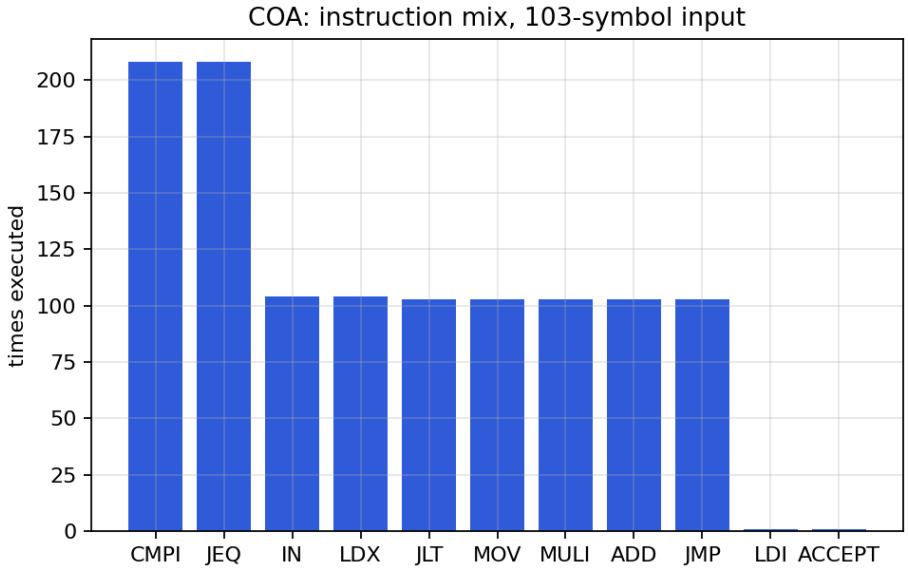


Figure 3: Dynamic instruction mix for a 103-symbol input

The mix is dominated by compare and branch instructions (CMPI and JEQ make up about 36% of all executed instructions): typical control-heavy code. Only one instruction per symbol touches memory (`LDX`), and it is the one that uses indexed addressing.

Testing

Table 9: Automated test results (python -m unittest)

Test group	Tests	Result
NFA / DFA / minimisation	5	PASS
CPU, assembler, instruction cycle	7	PASS
End-to-end vs Python `re` (incl.	4	PASS

3,000-case fuzz)		
Regex parser (postfix, classes, error handling)	5	PASS
Tarjan & Kosaraju SCC, dead states, finiteness	7	PASS
Total	28	**ALL PASS**

The CPU-specific tests check encode/decode round trips, arithmetic and flag behaviour, every addressing mode, exact cycle accounting, presence of all six micro-steps (T0–T5) in a trace, error handling (undefined opcode, out-of-range memory access, assembler errors) and the cycle-limit guard against infinite loops.



*Figure 4: Web application: compilation pipeline summary for (a|b)*abb*

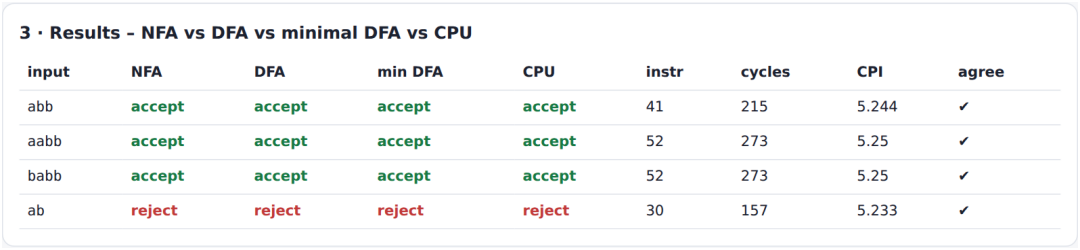


Figure 5: Web application: all four engines agree; instructions, cycles and CPI per input

Chapter IV – Conclusion

This work shows that the instruction cycle can be implemented, observed and measured in a few hundred lines of code. The simulated CPU fetches, decodes and executes real 32-bit machine words, exposes every register transfer in a trace, supports immediate, direct, register and indexed addressing, and accounts for clock cycles with a simple but explicit model. A program generated automatically from a regular expression ran correctly on it, and its measured cost (41 + 58n cycles, CPI ≈ 5.27) matched the analytical prediction exactly.

The main limitation is that the CPU is not pipelined: every instruction occupies the machine for its full 5–7 cycles. Natural extensions are a five-stage pipeline with hazard detection, a hardwired or microprogrammed control unit, a cache in front of the transition table, and a richer ISA (for example a conditional-move or table-lookup instruction) that would shorten the 11-instruction inner loop.